

MEEN 433/667 Mechatronics

Laboratory Introduction Monitor Program

Software license

- IDE and Assembler: MPLAB (free)
http://www.microchip.com/stellent/idcplg?IdcService=SS_GET_PAGE&nodeId=146&dDocName=en019469&part=SW007002
- C Compiler: PIC MCU C Compiler (commercial)
<http://ccsinfo.com/content.php?page=compilers>
- Monitor Program (free) <http://www.seanet.com/~karllunt/picload.htm>
- RS232C Terminal Program: Hyper Terminal (Imported from Windows XP operating system)

1. Microcontroller

New microcontrollers are introduced from Microchip (check course website and <http://www.microchip.com> for the latest information including data sheets, application notes.). We are going to use **PIC16F877** (40-pin DIP). It includes 10-bit A/D, RS232C, SPI, I²C, PWM generator, and many extra functions. It has flash program memory so that we can easily program/erase the program. Another advantage is that it is a RISC (Reduced Instruction Set Computers) processor. It runs on up to 20-MHz clock.

2. 2840 Development Board

This development board has a PIC16F877 with RS232C and USB serial communication circuits installed. There is a voltage regulator (7805) on board with a 10-MHz oscillator for clock generation. This board provides ISP (In System Programming) feature. You need to connect the header to necessary pins in order to use RS232C. It is necessary to connect CPU RX (RC7) to RS232C RX, CPU TX (RC6) to RS232C TX. Use the 9-pin straight through cable (9-pin Female to 9-pin Male) for RS232C communication, or the USB standard-b connector for USB communication. The monitor program (10-MHz, 9600-bps version) is programmed on PIC16F877. You can download your source code (in HEX file) using the Windows Hyper Terminal program.

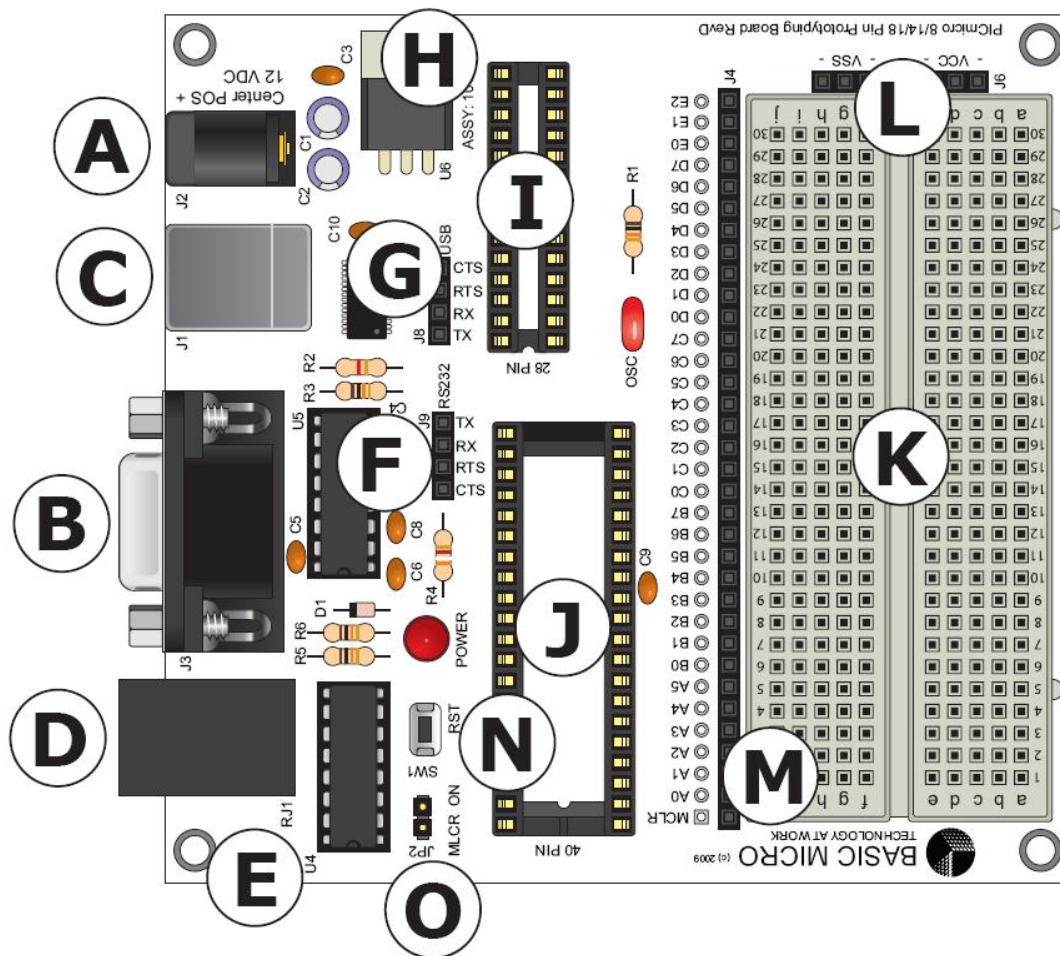


Fig. 1. 2840 Development Board Hardware Overview¹

- A:** 2.1mm Power Jack Center Positive, 6 to 9VDC
- B:** DB9 RS232 connector.
- C:** USB connector. (FTDI) USB chip.
- D:** RJ11 connector for in circuit programming. Works with ISP-PRO or Microchip programmer.
- E:** Auto isolation circuit for in circuit programming.
- F:** MAX232 circuit.
- G:** FTDI USB to Serial microcontroller.
- H:** 5VDC up to 500mA power supply circuit.
- I:** Socket for 28 pin PICmicros. Only one chip can be populated at a time.
- J:** Socket for 40 pin PICmicros. Only one chip can be populated at a time.
- K:** Solderless bread board.
- L:** Female header with ground and power.
- M:** Female header for access to all PICmicro I/O.
- N:** Reset button. Pulls MCLR to ground on all PICmicro sockets.
- O:** MCLR disconnect header. Jumper must be installed to program PICmicros. Remove for access to MCLR pin.

¹ 2840 Development Board Data Sheet

3. Lab X-1 Experimental Board

The LAB-X1 contains the circuitry required by the PICmicro to operate: 5-V power supply, oscillator, reset circuit, as well as additional application circuits. The crystal controlled oscillator includes jumpers to set speeds of 4, 8, 10, 12, 13.32, 16, and 20 MHz.

Application circuits include a switch matrix, potentiometers, LEDs, LCD module, serial EEPROMS², real-time clock³, temperature sensors⁴, servo connectors, RS232 interface, RS485 interface⁵, IR interface⁶, and speaker.

The LAB-X1 includes in-circuit programming connectors so the resident PICmicro MCU may be reprogrammed on the fly (requires flash device) using programmers that support this feature. Because the LAB-X1 itself is not a programmer, a separate PICmicro microcontroller programmer, such as the EPIC Programmer is required to program a PICmicro MCU. All of the PICmicro MCU I/O pins are brought out to headers next to a 40-pin ZIF socket. This allows connection to off-board circuits as well as allowing on-board circuits to be connected to other pins, if desired. The LAB-X1 is designed to work with 40-pin PICmicro MCUs, but may be jumpered to work with smaller devices. It is recommended the LAB-X1 be used with the 40-pin PIC16F877 (available separately). This flash device has 8 K of code space, 33 I/O pins, 8 ADC channels and may be reprogrammed again and again.

The complete Lab X-1 experimental board manual and the sample BASIC codes can be downloaded at <http://www.melabs.com/downloads/labx1.zip>.

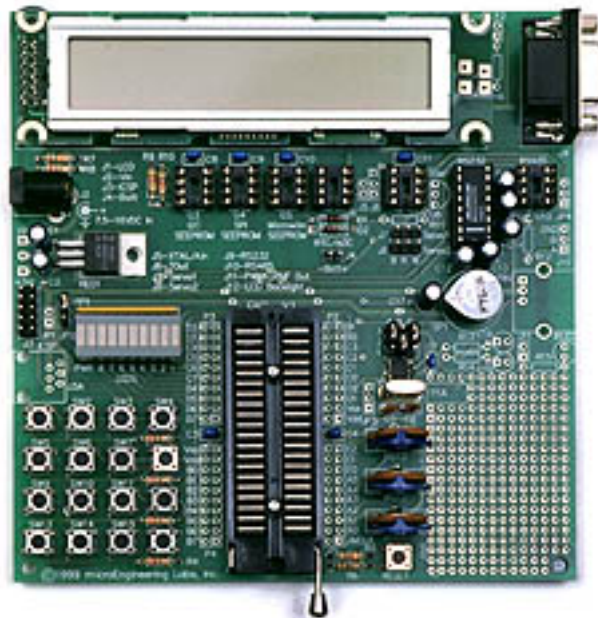


Fig. 2. Lab X-1 Development Board

^{2, 3, 4, 5, 6} Optional

4. Development Software

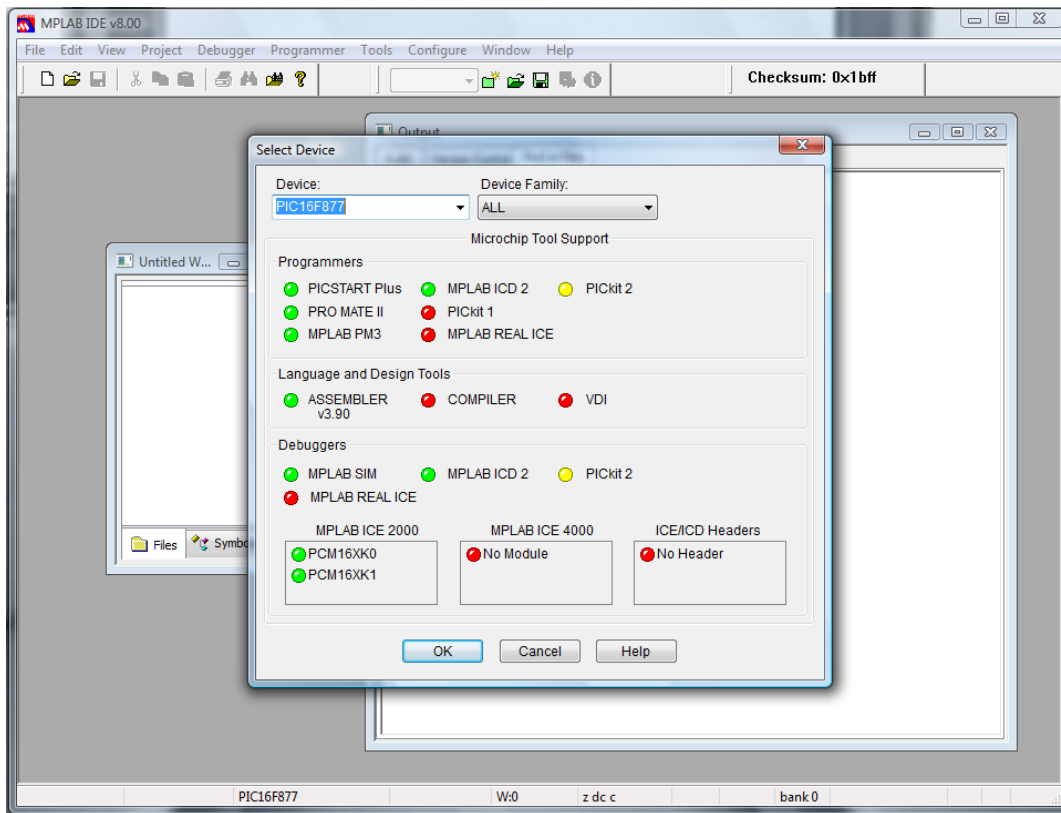
4.1. C compiler (<http://ccsinfo.com/content.php?page=compilers>)

This compiler has no built-in editor. It should be used with MPLAB in order to edit the source code.

4.2. MPLAB Setup

(http://www.microchip.com/stellent/idcplg?IdcService=SS_GET_PAGE&nodeId=1406&dDocName=en019469&part=SW007002)

Once MPLAB and PIC C compiler are installed, open MPLAB. On the menu bar, choose **Configure>Select device** and set **Device** to **PIC16F877**.

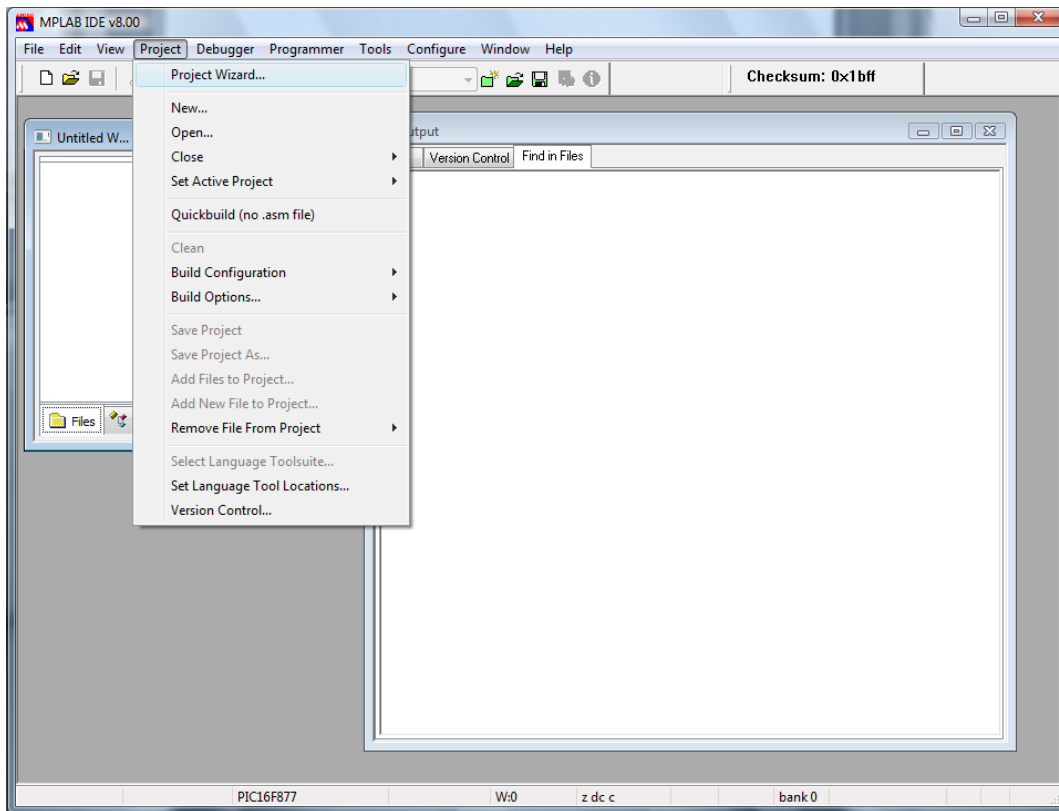


5. Monitor Program

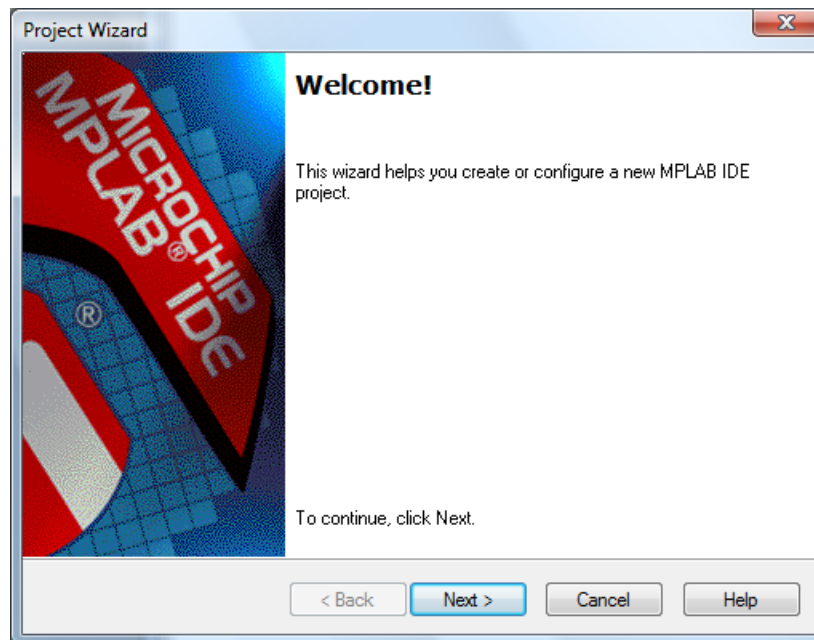
The monitor program programmed in PIC16F877 is based on the article from *Nuts & Volts* (pp. 63-68, April 2001) magazine. (<http://www.seanet.com/~karllunt/>). The original program is `picboot.zip` (`BOOTLDR3.ASM` with `BANK.MAC`). `boot877.asm` is the modified version for a 10-MHz OSC.

5.1. Editing and Compiling with MPLAB.

1. Start MPLAB's Project Wizard by selecting **Project->Project Wizard**:

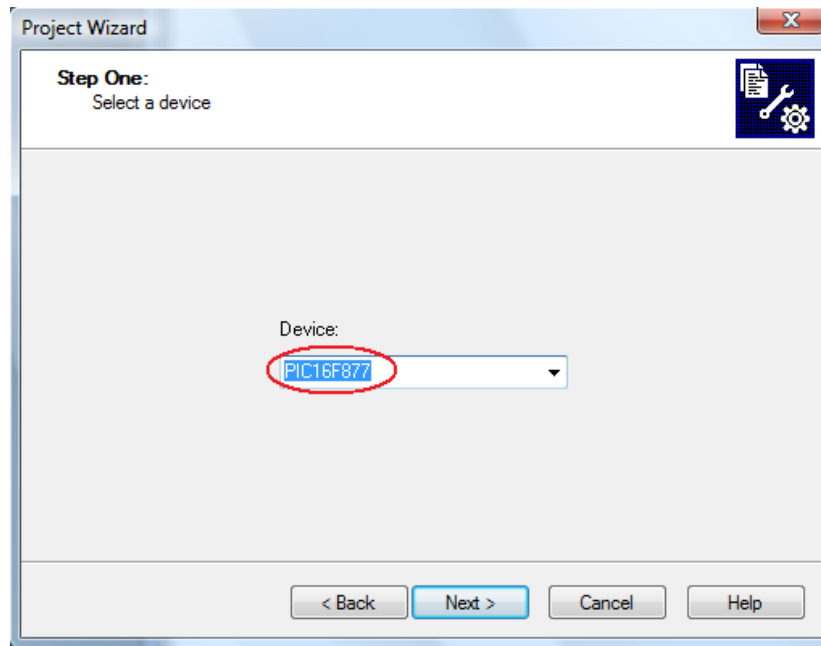


You will then be greeted with the Wizard splash screen:



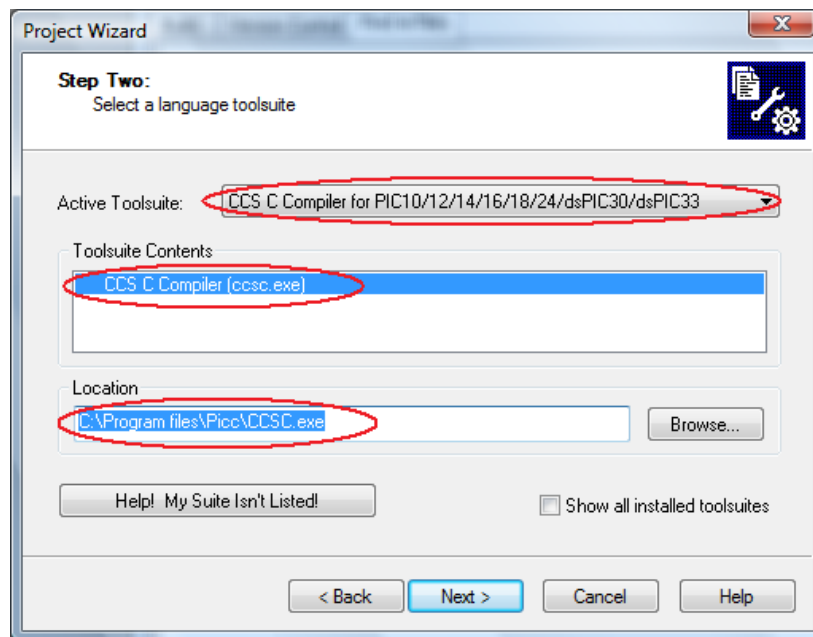
Press NEXT to continue.

2. Select the device to be **PIC16F877** and press NEXT to continue.



3. The next screen of the wizard will ask you which build suite you wish to use for your project. Select the tool suite labeled **CCS C Compiler for PIC10/12/14/16/18/24/dsPIC30/dsPIC33**. If you do not see this option then the plug-in was not installed correctly. **MAKE SURE YOU CHOOSE THE RIGHT C COMPILER SUITE AND GIVE THE CORRECT DIRECTORY PATH.**

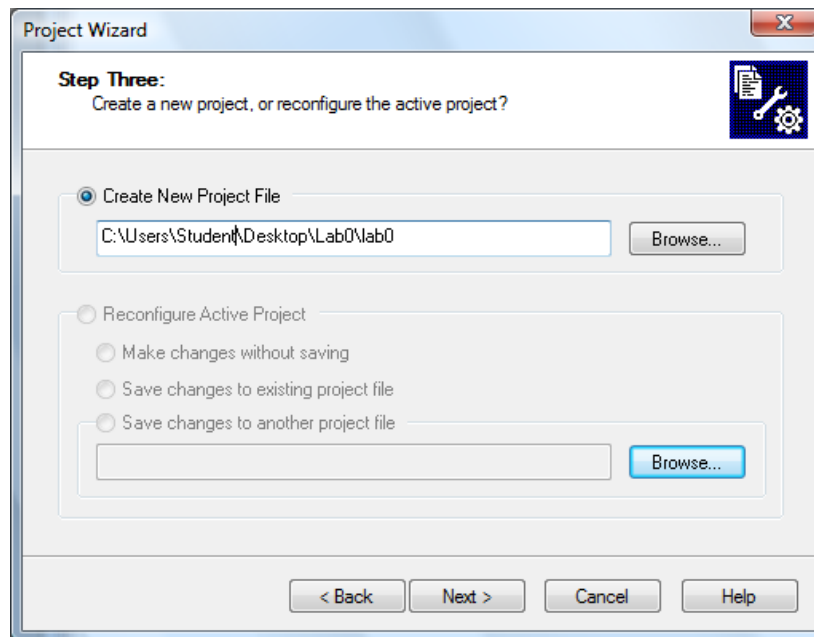
Your tool-suite select screen should look like this:



Verify that the location field is where you installed the compiler. Press NEXT to continue.

4. The next step of the New Project Wizard will ask you what to call the project and where to save the associated project files.

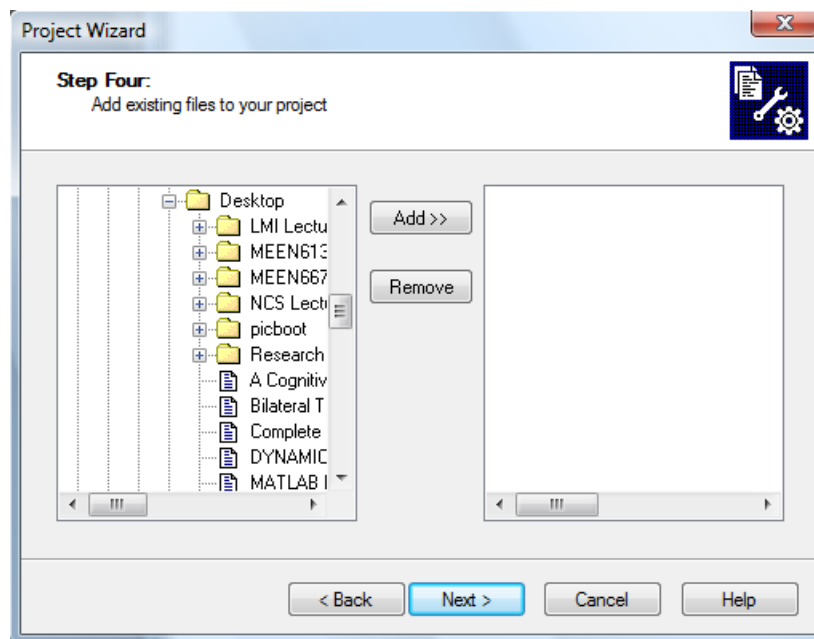
When done, your screen should look like this:



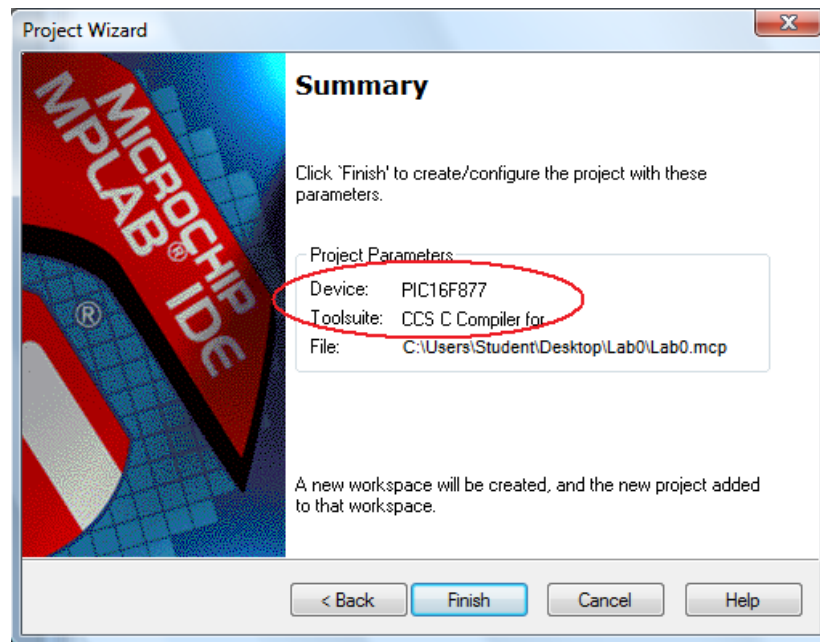
Press NEXT to continue.

5. The next step of the New Project Wizard asks you what files you wish to include with the project. In our lab session, just press NEXT to continue.

When done, your screen should look like this:



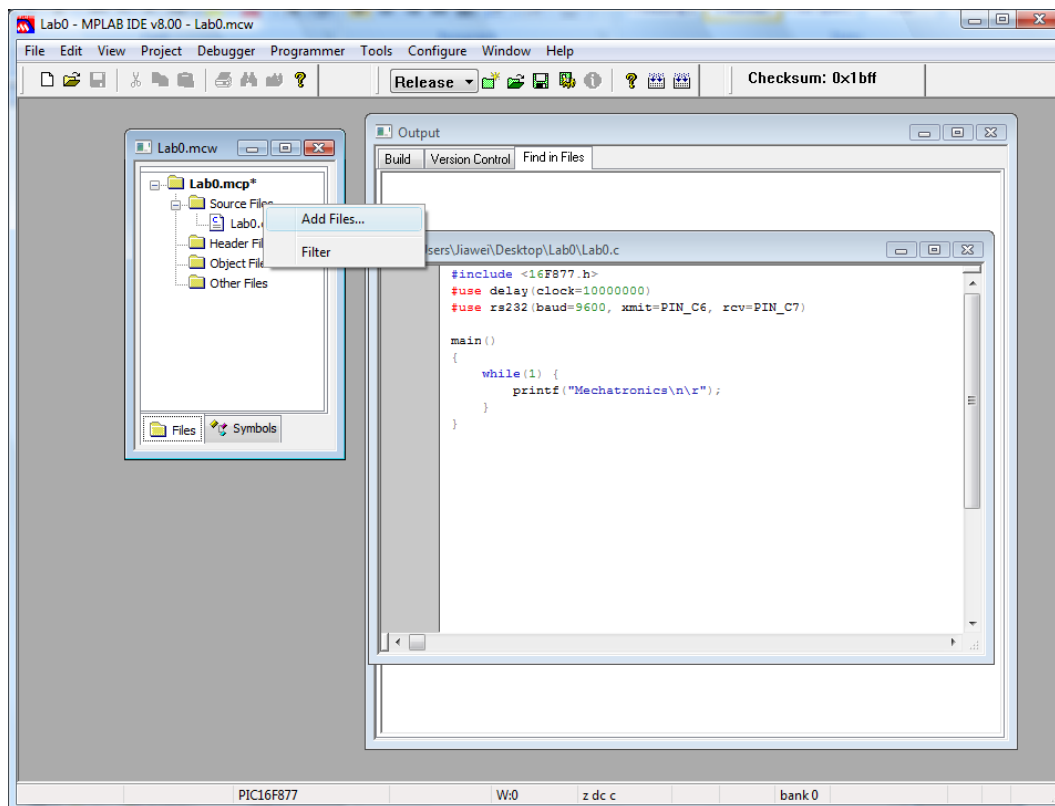
6. MPLAB Project Wizard will now give you a summary:



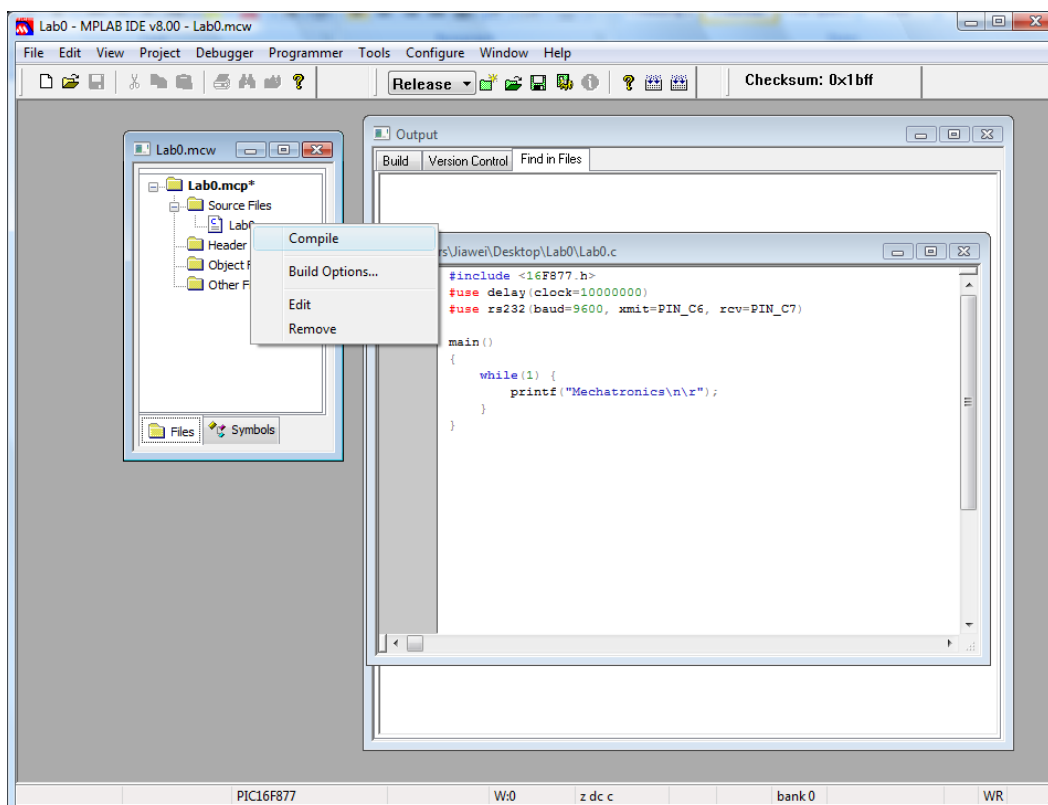
Press FINISH to continue. **Please double-check that your device is PIC16F877 and the Toolsuite is CCS C compiler.**

7. The Project Wizard will now close and MPLAB® will be back to the normal mode. In the Project Window on the left side you will see the <Project Name> files tree. Right-click the *Source Files* button and add your *.c file to the source files. Your *.c file can be written in any text editor and saved as *.c.

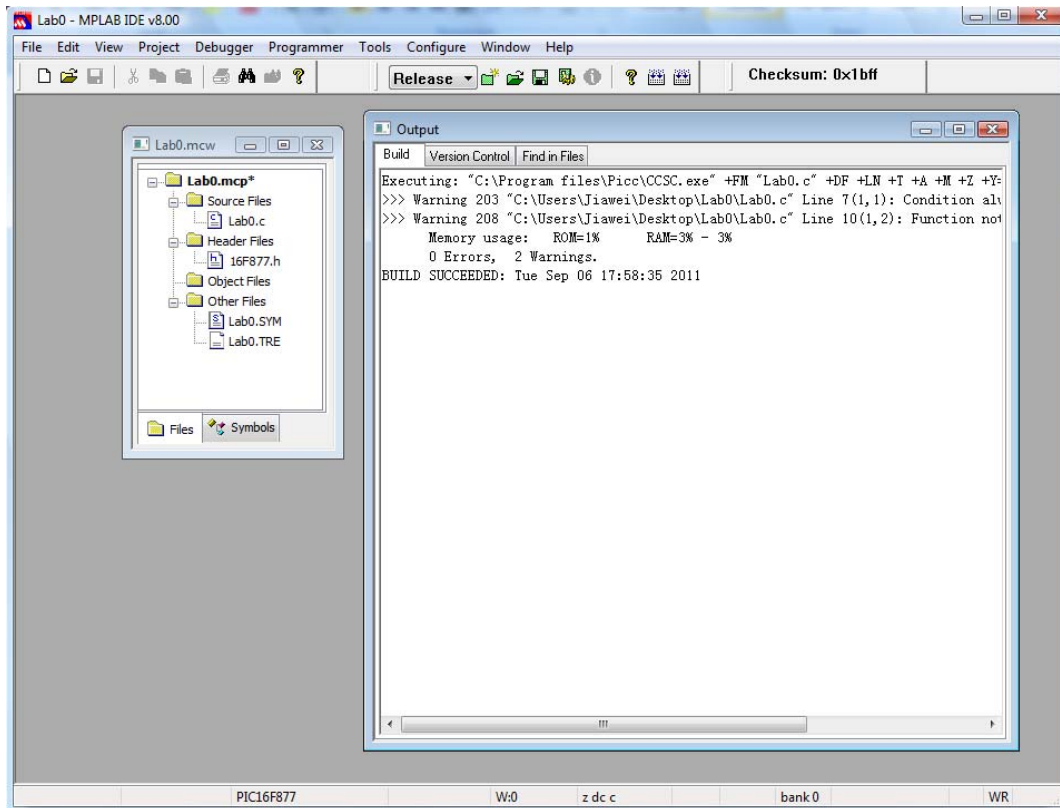
Your screen should look like this:



8. Now that the project is completed, you can compile it by right-clicking your *.c file and click on Compile:



The output window will display the compiler message, and it will say BUILD SUCCEEDED:



Now, you should have your *.hex file in the same directory as your *.c file. If there were any syntax errors, the output window would display the error messages.

6. Connection of 2840 board/Lab X-1 board

The procedure to connect the 2840 board to the PC is as follows.

For RS232C connection:

- Connect the DB9-Female side of the serial cable to the DB9 connector (B in Fig.1) of the 2840 board. The other end of the serial cable goes to the PC serial port.
- Connect the Power supply to the 2840 board (A in Fig.1).

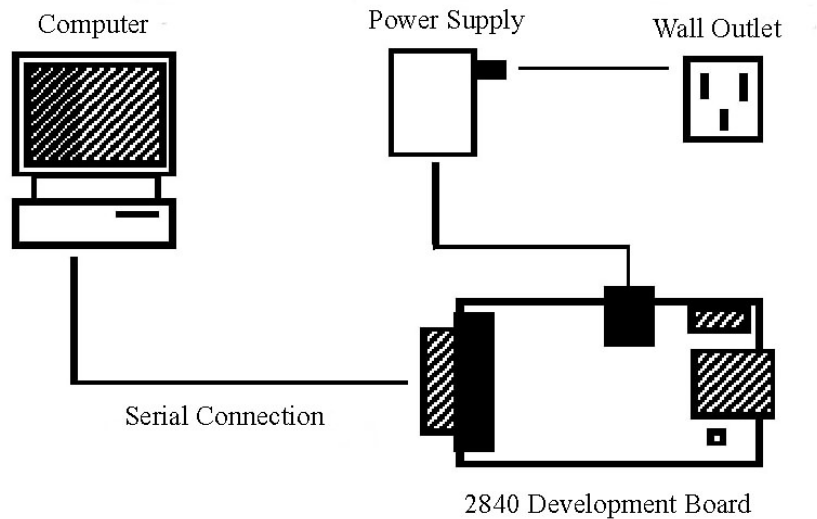


Fig. 3. Connection of 2840 board to PC

The procedure to connect the Lab X-1 board to the PC is as follows.

- Connect the DB9-Female side of the serial cable to the DB9 connector of the Lab X-1 board. The other end of the serial cable goes to the PC serial port.
- Connect the Power supply to the Lab X-1 board.

7. Windows Hyper Terminal Setting

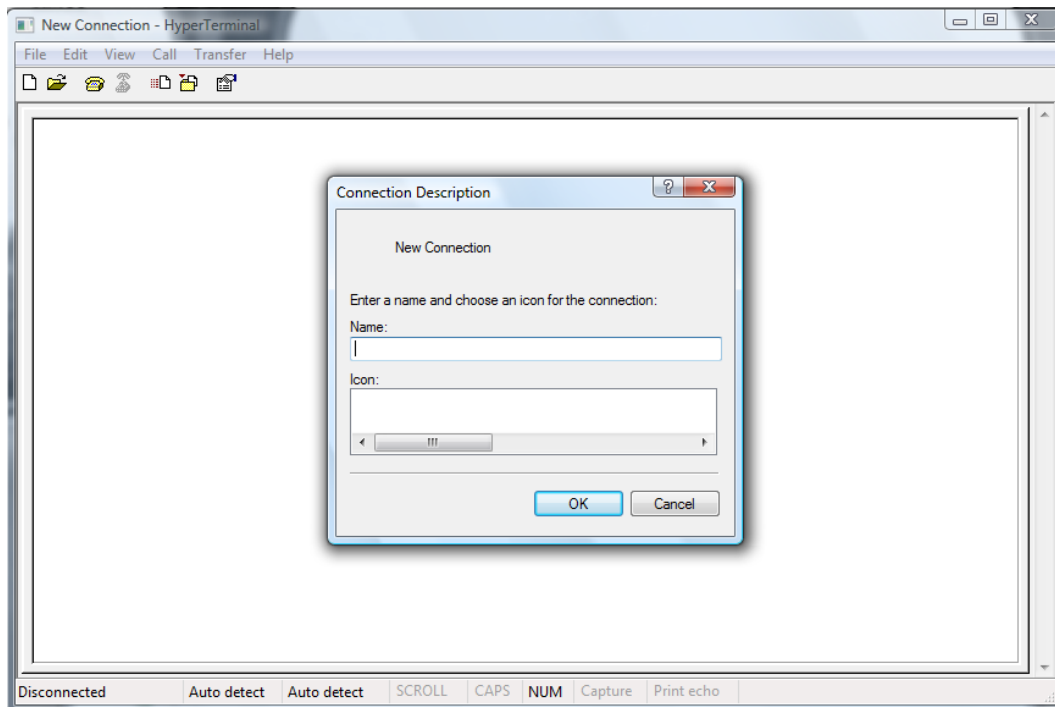
For the communication between the PC and the 2840 board or Lab X-1 board, we use Windows Hyper Terminal program.

Find the Hyper Terminal program at

C:\Program Files\hypertrm\hypertrm.exe

Choose NO when it asks for if you want to set Hyper Terminal as your default Telnet connection program. Check “Do not show this dialogue again” for your later convenience.

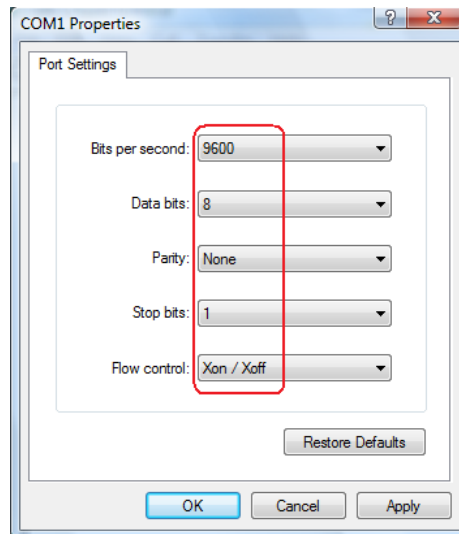
Enter the name for your new connection and click OK. Your screen should look like this:



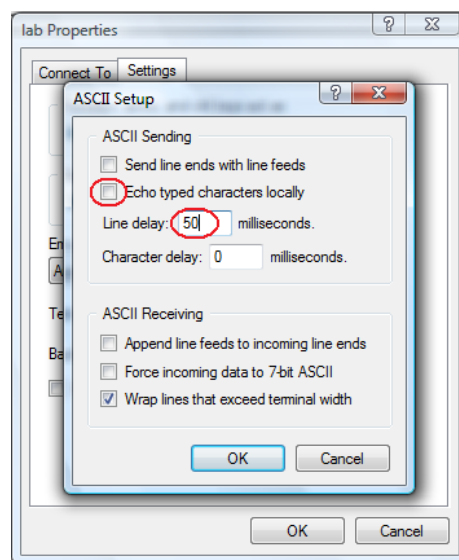
Choose **COM1** as your connection port as you will see:



Set up your connection properties as 9600, 8, None, 1, and Xon/Xoff. Click **OK**.



Also, in your new connection, set **Files > Properties > Settings > ASCII setup > Line Delay** as 50 msec, and **deselect Echo typed characters locally**. Click OK, and OK again.



Press ENTER, you will see a command prompt, Lee>. Now, you have setup your connection for the lab purpose.

Remember to save the current connection as a *.ht file in its default path for your later convenience. Just simply click Files > Save.

Now, you can download your executable program (*.hex) to the PIC16F877 using file transfer function of the hyper terminal. **Transfer > Send Text File** and select the correct directory and type in the file name. Once the download is completed, you can run the user program by typing **g!** and pressing Enter. You can terminate the program by pressing the **RESET** button on the 2840/Lab X-1 board. If you have terminated the program and want to re-run it, type **g!** and press Enter. You do not have to download the HEX file again in order to run the same program.

MEEN 433/667 Mechatronics

Laboratory #1 Microcontroller Architecture

All microprocessor and microcontroller systems are based on similar building blocks. These are shown in Fig. 1 and consist of the following:

- CPU (central processing unit) - the part that does all logic and arithmetic functions
- RAM (random access memory) - storage for programs and/or program variables
- EPROM (erasable-programmable read-only memory) - read-only parts of programs
- I/O (input/output)- connection to external devices

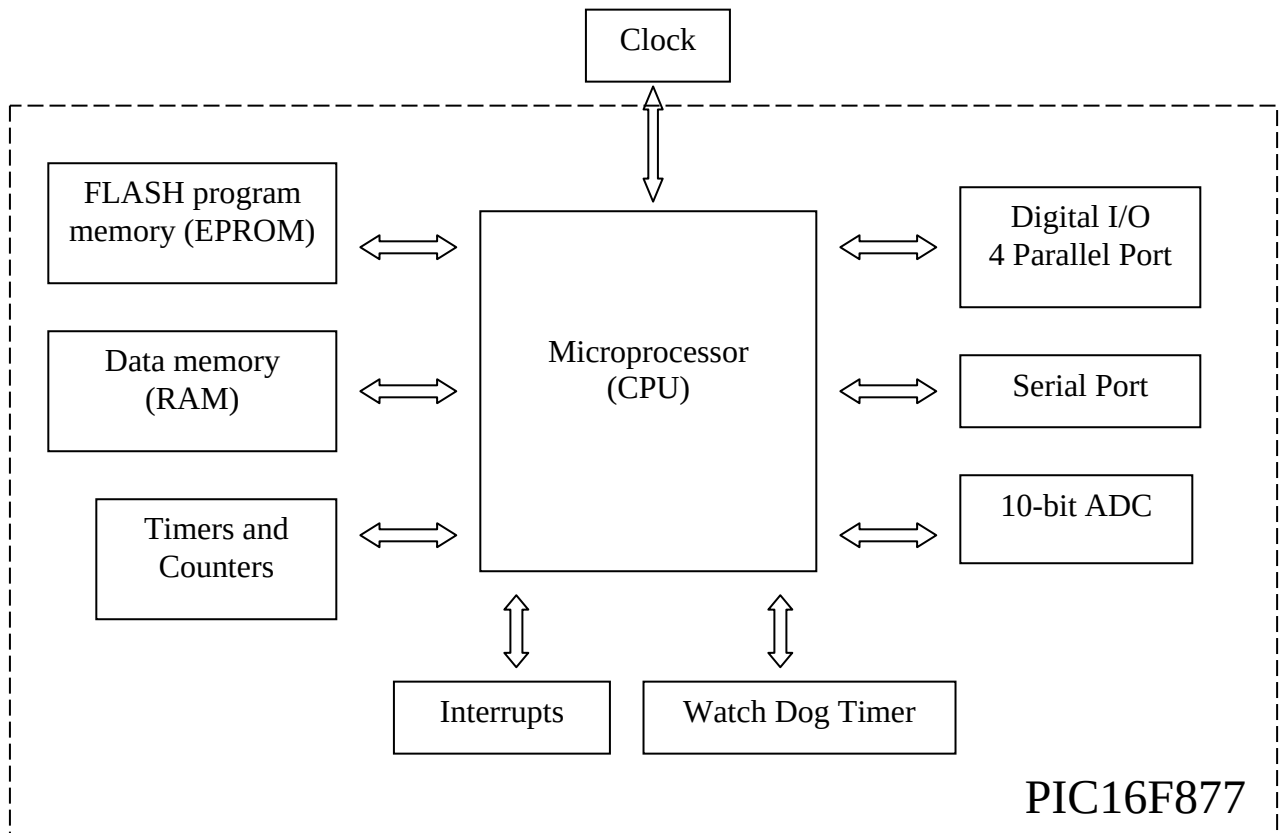


Fig. 1. PIC 16F877 architecture

The CPU or microprocessor is the core component of any microcomputer and it requires external components such as the ROM, RAM, I/O, etc. to accomplish its purpose. A microcontroller contains all the important features placed on one chip along with several other useful peripherals as ADCs (analog-to-digital converters), timers, counters, and I/O ports. The microcontroller-based system requires no additional circuitry except clock input. The difference between the microprocessor and the microcontroller arises because of their different end-usage. The microcontroller that will be investigated is the PIC16F877, which is at the upper end of the mid-range series of the microcontrollers developed by MicroChip, Inc. It is characterized by RISC (reduced instruction-set computer) architecture instead of the CISC (complex instruction-set computer) architecture used, for example, by the Motorola 6811.

The other peripheral features of 16F877 are:

- Timer0: 8-bit timer/counter with 8-bit prescaler
- Timer1: 16-bit timer/counter with prescaler, can be incremented during SLEEP via external crystal/clock
- Timer2: 8-bit timer/counter with 8-bit period register, prescaler and postscaler
- Two Capture, Comparator, PWM modules
 - Capture is 16-bit with maximum resolution of 12.5 ns
 - Comparator is 16-bit with maximum resolution of 200 ns
 - PWM is with maximum resolution of 10 bits
- 10-bit multi-channel Analog-to-Digital converter (ADC)
- Universal Synchronous Asynchronous Receiver Transmitter (USART/SCI) with 9-bit address detection
- Parallel Slave Port (PSP) 8-bits wide, with external RD, WR and CS controls (40/44-pin only)

The PIC series of microcontrollers are RISC-based processors with an accumulator (also called the working register, W). The microcontroller has a program memory data bus and a data memory data bus. Separate buses mean that simultaneous access of program and data can be done, which gives a greater bandwidth over the traditional architecture. Separating the program and data memories allow instructions to be sized differently than the 8-bit wide data word. This separation means that the instruction words can be ideally sized for the specific CPU/application. The PIC OpCodes for the mid-range processors are 14-bits wide, and the 14-bit wide program bus fetches an instruction in a single cycle.

1. Peripheral Interfaces

The PIC16F877 has several built-in peripheral interface features which are multiplexed with the port pins, namely: a Parallel Slave Port (PSP), a 10-bit, 8-channel A/D converter, two Capture/Compare/PWM modules for external triggered actions, two serial communication peripherals. Each of these features could be implemented using a PIC without peripheral interfaces.

1.1. Parallel Bus Interfaces and the Parallel Slave Port (PSP)

A bus is a collection of lines used to transfer information between two devices. Typically a bus has selection/address lines, data lines, and read/write control lines. The bus master is responsible for initiating bus transactions, and may also provide the clock or enable signals for the external device. A parallel bus, for which the PIC is the bus master, may be simulated using the I/O ports. A bus may be synchronous or asynchronous, and could be used for the control of an external device. Note that in any such scheme, the software must be written so that the read/write timing specifications of the external device are met that is called the driver of the device. Where there are insufficient port pins available on the PIC for all the bus lines, special-purpose interfaces, or even general purpose serial-to-parallel converters may be employed. The PIC then pulses out the bits one at a time, before triggering the bus operation.

The PIC microprocessor may also act as a slave on a parallel bus controlled by another microprocessor. In that case, the Parallel Slave Port should be used, and the master should observe the PIC's response timing. The CPU responds to external signals of the PSP in Q2 of the instruction cycle.

1.2. A/D Converter

The built-in analog-to-digital converter (ADC) of the PIC16F877 has 8 channels, and these are multiplexed across ports A and E. The first 5 channels are on Port A (Pin 2-5 & 7) and the rest on Port E (Pin 8-10). The ADCON0 register controls the operation of the A/D module. The ADCON1 register configures the functions of the port pins. The port pins can be configured as analog inputs, voltage reference, or as digital I/O.

The A/D module has high and low voltage reference inputs that are software selectable to some combination of VDD (device voltage), VSS (ground reference), RA2, or RA3. Ideally, the digital output value is the fraction of the difference between the reference-voltages in which the analog voltage falls. The A/D converter produces a 10-bit result.

$$Output = 2^{10} * \frac{V_{AN} - V_{ref}^-}{V_{ref}^+ - V_{ref}^-}$$

After the A/D module has been configured as desired, the selected channel must be acquired before the conversion is started. The analog input charges a sample and hold capacitor. There is a minimum acquisition time which must be observed, to ensure that the capacitor charges to within 0.5 LSB of the actual input voltage. This is estimated in the data sheet as a worst case value of 20 μ s, however this value is dependent on the temperature, the source impedance, and the supply voltage related switching resistance. For alternate conditions this acquisition time may be reduced. During conversion, the sample and hold capacitor is disconnected from the input, and the converter then generates a digital result of the analog level on the capacitor via successive approximation.

1.3. Compare/PWM

In Compare mode, the 16-bit CCPR1 register value is constantly compared against the TMR1 register pair value. When a match occurs, the RC2/CCP1 pin is:

- Driven high
- Driven low
- Remains unchanged

Pulse-Width-Modulation (PWM) is one of the means by which a digital value can be converted into an analog output. One of the very important applications of this function is to control the speed of a DC motor by digital control without use of any digital-to-analog converter (DAC). An operation of automatic toggling at a predefined signal frequency takes place. For a specified part of the time-period the output is SET (5 V) and for rest of the part it is RESET (0 V). The ratio of these timings is called the duty-cycle of the output signal. The duty-cycle may be expressed either in time, or as a percentage of the PWM period. The average voltage/power delivered by such a signal will be proportional to the fraction of the “on” time. The choice of PWM frequency depends on two factors. First, the frequency must be high enough that the device does not visibly/audibly/mechanically go on/off. Second, the frequency must be low enough that within the PWM period, there are sufficient “steps” available to express all the digital values, which will be converted.

1.4. Serial Communication

Serial Communications (even more so than Parallel bus transactions) are very dependent on timing; in order to pass large quantities of data, individual bits must be switched on the output lines at fast rates. Serial communication protocols define the means in which communication will take place, and define:

- Number of signal wires, and the voltage tolerances
- Transaction timing (therefore unlike the parallel bus the timing is independent of the particular peripheral) synchronous/asynchronous.
- The form of error detection/correction to be used (if any). This can be applied to single data-words (parity, start/stop bits) as well as data frames (SOF/EOF, header (Packet ID, Addressing) trailer (Checksum, ACK),
- The form of synchronization to be used if any {handshaking; XON-XOFF; arbitration/collision detection; master clock signals

It is clear that in serial communication, a lot more than the actual data to be transmitted is crossing the lines. The PIC16F877 includes peripherals that will independently perform serial communications: The Master Synchronous Serial Port (MSSP) for synchronous (I2C and SPI) communications, and a Universal Synchronous Asynchronous Receiver Transmitter (USART) for synchronous/asynchronous serial communications (RS232).

Experiment

- Connect the power supply and the serial cable to the 2840 Development Board.
- Run Hyper Terminal program.
- Check whether you get the prompt when you push ENTER key on PC keyboard.
- Run MPLAB.
- Type the following source code and save it as `lab2840.c`.

```
#include <16F877.h>
#use delay(clock=1000000)
#use rs232(baud=9600, xmit=PIN_C6, rcv=PIN_C7)

main()
{
    while(1) {
        printf("Mechatronics\n\r");
    }
}
```

- Compile `lab2840.c` and check whether you have any error and `lab2840.hex` file is generated.
- Download the `lab2840.hex` file to the development board and run it.
- The following sample code displays the ASCII value of the keyboard input to the hyper terminal in *Hexadecimal* number format. Try different numbers and see its ASCII value. Also try characters (lower, upper and special) and check with the ASCII table on the following page.

```
#include <16F877.h>
#use delay(clock=1000000)
#use rs232(baud=9600, xmit=PIN_C6, rcv=PIN_C7)

main()
{
    int b;

    while (1) {
        b = getc();
        printf("The ASCII value is %x \n\r", b);
    }
}
```

Assignment

- Modify the previous code so that it can display (on HyperTerminal) the sum of two single-digit integers (any of 0–9) you typed using the PC keyboard. Here are suggested steps:

1. Use the `getc()` C function (Refer to p. 96 of the PIC MCU C Compiler manual) to read the numbers you typed as ASCII characters.
2. Convert these ASCII characters to integer values.
3. Sum the two integers.
4. Display the sum using the `printf()` C function (Refer to pp. 135–137 of the PIC MCU C Compiler manual).

ASCII Table

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	##32;	Space	64	40	100	##64; @		96	60	140	##96; `	
1	1	001	SOH (start of heading)	33	21	041	##33; !		65	41	101	##65; A		97	61	141	##97; a	
2	2	002	STX (start of text)	34	22	042	##34; "		66	42	102	##66; B		98	62	142	##98; b	
3	3	003	ETX (end of text)	35	23	043	##35; #		67	43	103	##67; C		99	63	143	##99; c	
4	4	004	EOT (end of transmission)	36	24	044	##36; \$		68	44	104	##68; D		100	64	144	##100; d	
5	5	005	ENQ (enquiry)	37	25	045	##37; %		69	45	105	##69; E		101	65	145	##101; e	
6	6	006	ACK (acknowledge)	38	26	046	##38; &		70	46	106	##70; F		102	66	146	##102; f	
7	7	007	BEL (bell)	39	27	047	##39; '		71	47	107	##71; G		103	67	147	##103; g	
8	8	010	BS (backspace)	40	28	050	##40; (		72	48	110	##72; H		104	68	150	##104; h	
9	9	011	TAB (horizontal tab)	41	29	051	##41;)		73	49	111	##73; I		105	69	151	##105; i	
10	A	012	LF (NL line feed, new line)	42	2A	052	##42; *		74	4A	112	##74; J		106	6A	152	##106; j	
11	B	013	VT (vertical tab)	43	2B	053	##43; +		75	4B	113	##75; K		107	6B	153	##107; k	
12	C	014	FF (NP form feed, new page)	44	2C	054	##44; ,		76	4C	114	##76; L		108	6C	154	##108; l	
13	D	015	CR (carriage return)	45	2D	055	##45; -		77	4D	115	##77; M		109	6D	155	##109; m	
14	E	016	SO (shift out)	46	2E	056	##46; .		78	4E	116	##78; N		110	6E	156	##110; n	
15	F	017	SI (shift in)	47	2F	057	##47; /		79	4F	117	##79; O		111	6F	157	##111; o	
16	10	020	DLE (data link escape)	48	30	060	##48; 0		80	50	120	##80; P		112	70	160	##112; p	
17	11	021	DC1 (device control 1)	49	31	061	##49; 1		81	51	121	##81; Q		113	71	161	##113; q	
18	12	022	DC2 (device control 2)	50	32	062	##50; 2		82	52	122	##82; R		114	72	162	##114; r	
19	13	023	DC3 (device control 3)	51	33	063	##51; 3		83	53	123	##83; S		115	73	163	##115; s	
20	14	024	DC4 (device control 4)	52	34	064	##52; 4		84	54	124	##84; T		116	74	164	##116; t	
21	15	025	NAK (negative acknowledge)	53	35	065	##53; 5		85	55	125	##85; U		117	75	165	##117; u	
22	16	026	SYN (synchronous idle)	54	36	066	##54; 6		86	56	126	##86; V		118	76	166	##118; v	
23	17	027	ETB (end of trans. block)	55	37	067	##55; 7		87	57	127	##87; W		119	77	167	##119; w	
24	18	030	CAN (cancel)	56	38	070	##56; 8		88	58	130	##88; X		120	78	170	##120; x	
25	19	031	EM (end of medium)	57	39	071	##57; 9		89	59	131	##89; Y		121	79	171	##121; y	
26	1A	032	SUB (substitute)	58	3A	072	##58; :		90	5A	132	##90; Z		122	7A	172	##122; z	
27	1B	033	ESC (escape)	59	3B	073	##59; ;		91	5B	133	##91; [		123	7B	173	##123; {	
28	1C	034	FS (file separator)	60	3C	074	##60; <		92	5C	134	##92; \		124	7C	174	##124;	
29	1D	035	GS (group separator)	61	3D	075	##61; =		93	5D	135	##93;]		125	7D	175	##125; }	
30	1E	036	RS (record separator)	62	3E	076	##62; >		94	5E	136	##94; ^		126	7E	176	##126; ~	
31	1F	037	US (unit separator)	63	3F	077	##63; ?		95	5F	137	##95; _		127	7F	177	##127; DEL	

Source: www.asciitable.com

Experiment

- Connect the power supply and the serial cable to the Lab X-1 Experimental Board.
- Run Hyper Terminal program.
- Check whether you get the prompt when you push ENTER key on PC keyboard.
- Run MPLAB.
- Type the following source code and save it as `labX1.c`. This program will display the key number on the HyperTerminal and the Lab X-1 LCD display. Refer to PIC MCU C Compiler Manual and the Lab X-1-related datasheets (`labX1.pdf`, `labX1_DM2023.pdf`, and `labX1_HD4470U.pdf`) to understand the working of the functions used in the following program.

```
#include<16F877.h>
#include<stdlib.h>
#include delay(clock=1000000)
#include rs232(baud=9600, xmit=PIN_C6, rcv=PIN_C7)
```

```

#define ENA PIN_E1
#define RES PIN_E0
#define WRI PIN_E2

char display_data[3];
void write_code(unsigned char code);
void write_data(unsigned char code);
void lcd_init();
void display(int number);
void display_string(char *string);

main()
{
    int b[4], column = 0, row, sw;

    set_tris_b(0x0F);
    port_b_pullups(true);
    lcd_init();
    output_high(PIN_B4);
    output_high(PIN_B5);
    output_high(PIN_B6);
    output_high(PIN_B7);
    while (1) {
        for (column = 0; column <= 3; column++) {
            if (column == 0)
                output_low(PIN_B4);
            else if (column == 1)
                output_low(PIN_B5);
            else if (column == 2)
                output_low(PIN_B6);
            else if (column == 3)
                output_low(PIN_B7);
            b[0] = input(PIN_B0);
            b[1] = input(PIN_B1);
            b[2] = input(PIN_B2);
            b[3] = input(PIN_B3);
            for (row = 0; row <= 3; row++) {
                if (!b[row]) {
                    sw = column + 4*row + 1;
                    printf("SW%d is pressed. \n\r", sw);
                    display(sw);
                }
            }
            if (column == 0)
                output_high(PIN_B4);
            else if (column == 1)
                output_high(PIN_B5);
            else if (column == 2)
                output_high(PIN_B6);
            else if (column == 3)

```

```

        output_high(PIN_B7);
        delay_ms(50);
    }
}

void lcd_init()
{
    unsigned char code;
    char temp[40];

    code = 0x38;
    write_code(code);
    write_code(code);
    write_code(code);
    code = 0x06;
    write_code(code);
    code = 0x0C;
    write_code(code);
    delay_ms(4);
    code = 0x0C;
    write_code(code);
    code = 0x06;
    write_code(code);
    code = 0x01;
    write_code(code);
    delay_ms(2);
    code = 0x82;
    write_code(code);
    strcpy(temp, "Number input is:");
    display_string(temp);
}

void display(int number)
{
    unsigned code;

    code = 0xC7;
    write_code(code);
    if (number >= 10) {
        number=(number-10);
        display_data[0] = 0x31;
        display_data[1] = 0x30+number;
        display_data[2] = NULL;
    }
    else {
        display_data[0] = 0x30;
        display_data[1] = 0x30+number;
        display_data[2] = NULL;
    }
    display_string(display_data);
}

```

```

}

void display_string(char *string)
{
    while (*string) {
        write_data(*string++);
    }
}

void write_code(unsigned char code)
{
    output_low(RES);
    output_low(WRI);
    output_low(ENA);
    output_d(code);
    output_high(ENA);
    output_low(ENA);
    output_high(WRI);
    delay_ms(1);
}

void write_data(unsigned char code)
{
    output_low(WRI);
    output_low(ENA);
    output_high(RES);
    output_d(code);
    output_high(ENA);
    output_low(ENA);
    output_high(WRI);
    delay_ms(1);
}

```

- Compile `labX1.c` and check whether you have any error and `labX1.hex` file is generated.
- Download the `labX1.hex` file to the development board and run it.

MEEN 433/667 Mechatronics

Laboratory #8 Interface with Sensors

1. Infrared Range Sensor

You are going to measure the distance using an infrared (IR) sensor module. Read the AIRRS sensor manual (AIRRS.pdf) and the sensor datasheet (gp2d12datasheet.pdf). Manufacturer's application notes can be found at www.sharpsma.com. Establish the following connections with the 2840 development board. The sensor output is connected to CH0 of the A/D converter. Watch the colors of the three sensor wires and ensure you make all the connections correctly before power-on.

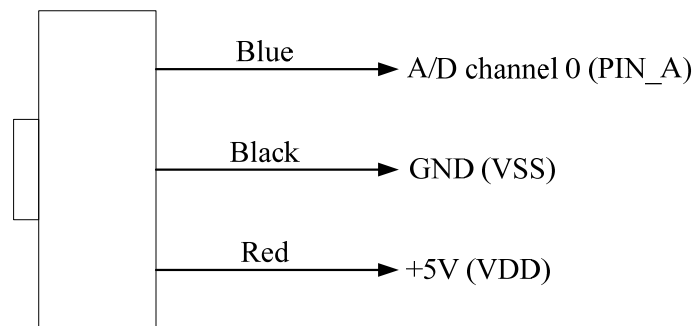


Fig. 1. IR sensor outputs

Assignment

Write a program which displays the voltage output from the IR sensor. Use the C code you wrote in the A/D converter lab in Lab 3 with replacing the potentiometer with this IR sensor. Record the values of distance (from 10 cm to 70 cm at every 5 cm) and the corresponding output voltage to calibrate this IR sensor.

2. Temperature Sensor

DS1620 is a digital thermometer from Dallas semiconductor with its output in the Celsius unit. You are going to measure the temperature using a DS1620. Read the sensor data sheet (ds1620.pdf). Manufacturer's application notes can be found at www.maxim-ic.com. There are three pins (DQ, CLK, /RST) to be interfaced with PIC16F877.

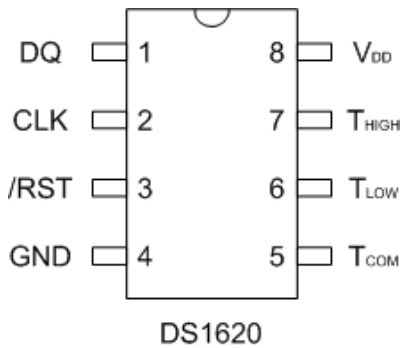


Fig. 2. DS1620 pin arrangement

Assignment

Connect V_{DD} (pin 8) to VDD and GND (pin 4) to VSS for power. For control/data communication, PIN_B0, PIN_B1, and PIN_B2 of the PIC16F877 microcontroller are used. Make wire connections as

DS1620	16F877
DQ (pin 1)	PIN_B0
CLK (pin 2)	PIN_B1
/RST (pin 3)	PIN_B2

Run the following sample program with your own functions of `GetByte()` and `PutByte()`. Refer to the DS1620 datasheet.

```
#include<16F877.h>
#use delay (clock=1000000)
#use rs232(baud=9600, xmit=PIN_C6, rcv=PIN_C7)

#define DQ PIN_B0
#define CLK PIN_B1
#define RST PIN_B2

unsigned char GetByte(void);
void PutByte(unsigned char d);

main()
{
    unsigned char Temp;
    unsigned char sign_bit;

    while (1) {
        output_low(RST);
        output_high(RST);
        PutByte(0xEE);
        output_low(RST);
        delay_ms(500);
    }
}
```

```

        output_high(RST);
        PutByte(0xAA);
        Temp=GetByte();
        sign_bit = GetByte();
        output_low(RST);
        Temp >>= 1; // Resolution is 0.5 degree in Celsius.
        printf("%d\n\r", Temp);
    }
}

unsigned char GetByte(void)
{
    // your function
}

void PutByte(unsigned char data)
{
    // your function
}

```

Following is a reference program for initialization, configuration check, end of conversion check. You do not have to run this program.

```

#include <16F877.h>
#define delay(clock=1000000)
#define rs232(baud=9600, xmit=PIN_C6, rcv=PIN_C7)

#define DQ PIN_B0
#define CLK PIN_B1
#define RST PIN_B2

unsigned char GetByte(void);
void PutByte(unsigned char data);

main()
{
    unsigned char Temp;
    unsigned char sign_bit;
    unsigned char data;

    output_low(RST);
    output_low(CLK);
    output_high(DQ);

    while (1) {
        output_low(RST);
        output_high(RST);
        PutByte(0xAC);
        Data = GetByte();
        output_low(RST);
    }
}

```

```

        if ((data & 0x03)!=0x03) {
            output_high(RST);
            PutByte(0x0C);
            PutByte(0x03);
            output_low(RST);
        }

        output_high(RST);
        PutByte(0x22);
        output_low(RST);
        output_high(RST);
        PutByte(0xEE);
        output_low(RST);

        do {
            output_high(RST);
            PutByte(0xAC);
            data=GetByte();
            output_low(RST);
        } while ((data & 0x80)!= 0x80);

        output_high(RST);
        PutByte(0xAA);
        Temp=GetByte();
        sign_bit=GetByte();
        output_low(RST);
        Temp >>= 1;
        printf("%d\n\r",Temp);
    }
}

unsigned char GetByte(void)
{
    // your function
}

void PutByte(unsigned char data)
{
    // your function
}

```

MEEN 433/667 Mechatronics

Laboratory #9 PWM and Interface with Actuators

1. Pulse Width Modulation with 2840 board

The 16F877 has two built-in PWM (with 10-bit resolution) modules. You can assign the PWM frequency and select the duty ratio.

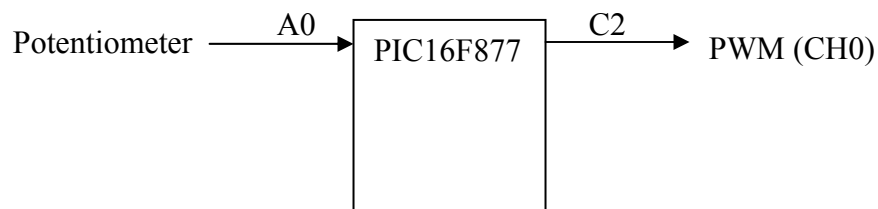


Fig. 1. Testing PIC16F877 PWM

Assignment

A sample program (pwm.c) is provided below. Connect the oscilloscope probe to the microcontroller PIN_C2 and observe the PWM signal while turning the potentiometer. Repeat with different PWM frequencies. This program displays the 10-bit PWM value on the screen, which is the 10-bit A/D value. Thus the duty ratio is

$$\frac{\text{PWMvalue}}{2^{10}} \times 100[\%] = \frac{\text{PWMvalue}}{1024} \times 100[\%].$$

```
#include <16F877.h>
#define ADC=10
#define delay(clock=1000000)
#define rs232(baud=9600, xmit=PIN_C6, rcv=PIN_C7)

main()
{
    char selection;
    long value;

    printf("\r\nFrequency:\r\n");
    printf("    1) 9.75 khz\r\n");
    printf("    2) 2.45 khz\r\n");
    printf("    3) 0.6 khz\r\n");
```

```

do {
    selection=getc();
} while ((selection<'1') || (selection>'3'));

setup_ccp1(CCP_PWM);

switch (selection) {
    case '1' : setup_timer_2(T2_DIV_BY_1, 255, 1);
               break;
    case '2' : setup_timer_2(T2_DIV_BY_4, 255, 1);
               break;
    case '3' : setup_timer_2(T2_DIV_BY_16, 255, 1);
               break;
}

setup_port_a(ALL_ANALOG);
setup_adc(adc_clock_internal);
set_adc_channel(0);
printf("%c\r\n",selection);

while(1) {
    value=read_adc();
    printf("PWM Command = %lu\n\r",value);
    set_pwm1_duty(value);
}
}

```

2. Low-pass-filtered PWM

Connect the 1-k Ω resistor and the 10- μ F capacitor as shown below. This is a simple RC low-pass filter. Note that the electrolyte capacitor has polarity.

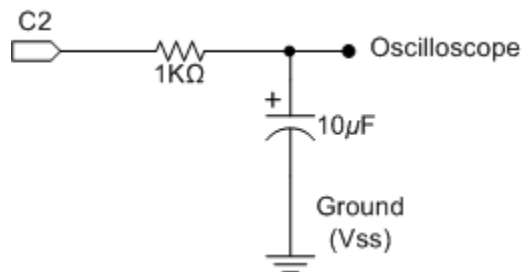


Fig. 2. First-order RC low-pass filter

Assignment

Calculate the cutoff frequency ($\frac{1}{2\pi RC}$ Hz) of the RC low-pass filter. Check the filtered PWM signal with different frequencies and different duty ratios. Using two

channels of the oscilloscope compare the filtered signal with the original PWM signal. Note that the cutoff frequency of the low-pass filter is fixed.

3. DC Motor Open-Loop Driving

Assignment

Use the ULN2803 Darlington transistor array to drive a DC motor in the unidirectional mode. Connect the circuit as in the diagram below. You must use the Agilent programmable power supply for $V_{cc}(\text{ext}) = 12 \text{ Vdc}$, and PIN 9 of ULN2830 must be connected to the Agilent programmable power supply's ground. Write a program (1) to start the DC motor and (2) to drive the DC motor at various speeds with PWM. Try various duty ratios for which you are able to observe the change in speed.

4. Solenoid Valve/Relay

Assignment

Use ULN2803 to drive a relay as shown in the diagram below. Run and stop the relay for 1 s each (PWM not required). Use the Agilent programmable power supply for $V_{cc}(\text{ext}) = 12 \text{ Vdc}$ and use a flyback diode to reduce the voltage surges when the relay switches off.

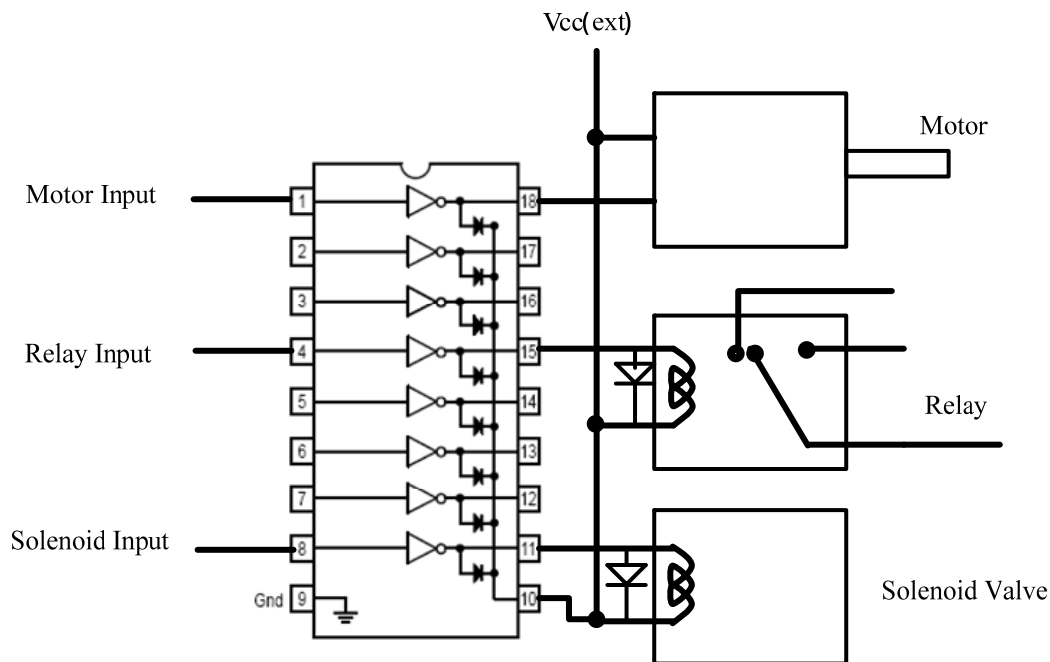


Fig. 3. Driving various devices using ULN 2803

MEEN 433/667 Mechatronics

Laboratory #10 DC Motor Closed-Loop Speed Control

The objectives of this lab are (1) design and implementation of a DC motor speed controller and (2) integration of a sensor, an actuator, and a real-time controller to build an intelligent mechatronic system. We will use a DC motor (Maxon A-max 110950 (refer to the datasheet: amax26.pdf)) and an encoder (Maxon Digital Encoder HEDS 5540 (refer to the datasheet: heds5540.pdf)).

1. DC Motor Modeling

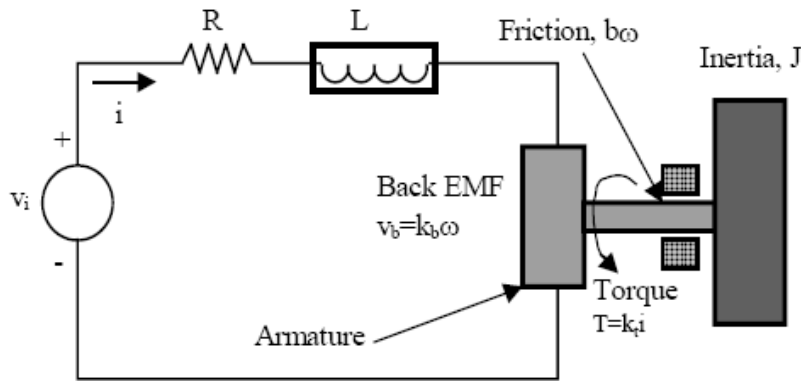


Fig. 1. DC motor system (from MEEN 364 Lab)

A schematic diagram of a DC motor system is shown in Fig 1. In a permanent-magnet DC motor, the developed torque is proportional to the armature current i_a . In other words,

$$T = K_t i_a, \quad (1)$$

where K_t is defined as the torque constant of the DC motor [N-m/A]. When the armature is rotating, a back-electromotive-force (emf) voltage v_b is produced that is proportional to the speed, ω .

$$v_b = K_b \omega, \quad (2)$$

where K_b is defined as the back emf constant of the DC motor [V-s/rad]. Applying the Kirchhoff's voltage law around the armature circuit, we obtain the electrical dynamics of the DC motor.

$$L \frac{di_a}{dt} + i_a R + v_b = v_i \quad (3)$$

If the external load is modeled as a moment of inertia and a damper as shown in Fig. 1, the mechanical dynamics of the DC motor with applying Newton's second law is

$$J \frac{d\omega}{dt} + B\omega = K_t i_a. \quad (4)$$

Combining (3) and (4), and neglecting the existences of the inductance L and the damping coefficient B , the DC motor dynamic model can be simplified as below.

$$J \frac{d\omega}{dt} + \frac{K_t K_b}{R} \omega = \frac{K_t}{R} v_i \quad (5)$$

A detailed modeling procedure can be found in Franklin and Powell (FP), Section 2.3.

2. Encoder for Speed Feedback

Refer to Labs. 4 and 5 to interface the encoder with the 16F877 microcontroller to measure the motor speed. Complete the wiring connection shown in Fig. 2. In this lab, we operate the DC motor unidirectionally (clockwise rotation seen from the right in Fig. 2), and you need not resolve the rotational direction. Thus, only Channel A is used although this encoder (Maxon Digital Encoder HEDS 5540) is a quadrature encoder like the mechanical encode we used in Lab. 5. Throughout this lab, leave the Channel B and Channel I pins open.

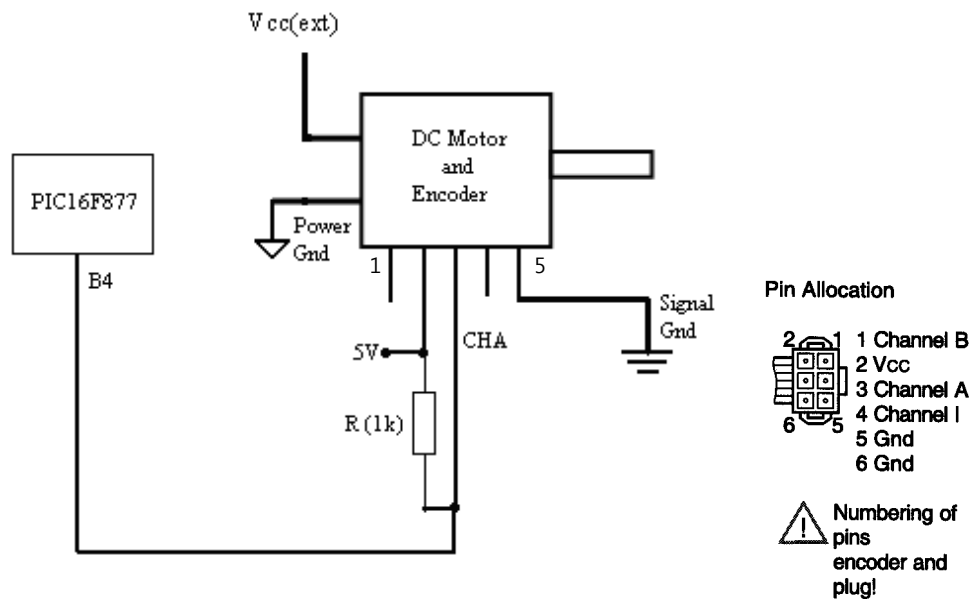


Fig. 2. Interfacing the encoder for speed feedback and connector pin allocation

A sample program is given below to display the speed of the DC motor in rps (revolution per second) measured by this encoder. In Lab. 5, we used polling and external interrupt (INT_RB) for the encoder interface. In the following code, the Timer1 interrupt is used with the sampling period of 0.209 s (i.e., the sampling frequency is 4.78 Hz). The factor 0.00552 used in the program is a parameter to make the output speed in the unit of rps.

```
#include <16F877.h>
#define delay(clock=1000000)
#define rs232(baud=9600, xmit=PIN_C6, rcv=PIN_C7)

long cnt = 0;
long speed = 0;

#define INT_TIMER1
Timer1_Interrupt()
{
    speed = cnt*0.00552;
    printf("Speed of DC motor is %ld rps. \n\r", speed);
    cnt = 0;
}

main()
{
    int b1, b2;

    setup_timer_1(T1_INTERNAL | T1_DIV_BY_8);
    enable_interrupts(INT_TIMER1);
    enable_interrupts(GLOBAL);

    while (1) {
        if (input(PIN_B4))
            b1 = 1;
        else
            b1 = 0;
        if (b1 != b2)
            cnt++;
        b2 = b1;
    }
}
```

Assignment

Vary $V_{cc(ext)}$ between 0 V and 16 V and observe the change in the motor speed. Use the Agilent programmable power supply for $V_{cc(ext)}$. *Never exceed $V_{cc(ext)} = 18\text{ V}$, the armature voltage rating of this motor.*

3. PWM for Motor Driving

Add the wiring connection shown in Fig. 3, which is almost the same as Fig. 3 of Lab. 9. Do not remove the wires that are connected to the encoder. Note one of the two DC motor armature leads must be connected to Pin 18 of ULN 2803. You must use the Agilent programmable power supply for $V_{cc(ext)} = 16 \text{ Vdc}$. *Never exceed $V_{cc(ext)} = 18 \text{ V}$, the armature voltage rating of this motor.*

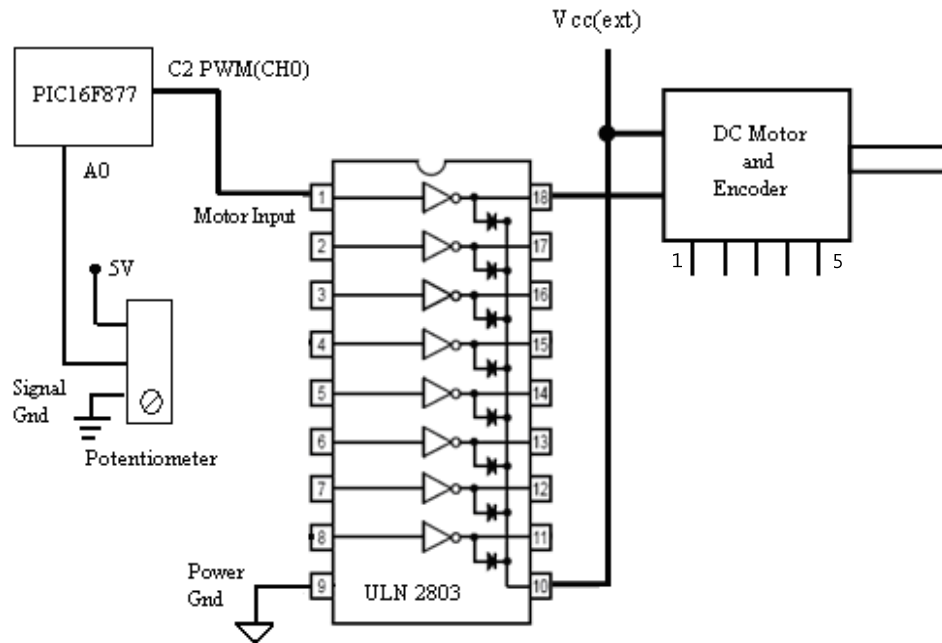


Fig. 3. DC motor system with PWM driving signal

Assignment

Repeat Section 3 of Lab. 9 with this new DC motor. After confirming the PWM's functionality, combine the PWM code you developed in Section 3 of Lab. 9 with the sample encoder code given in Section 2. Turn the potentiometer knob (instead of varying $V_{cc(ext)}$) to change and display the motor speed.

4. Speed Controller Design

To compensate for unknown parameter changes and model uncertainties in a dynamic system, closed-loop feedback control is often employed. Any feedback control can be broken into the following three sequential actions:

1. Measure the system output with a sensor.
2. Calculate the control effort based on a pre-designed control law with the measured sensor signal and the reference input.

3. Output this control effort through an actuator to make the system output track the reference input faithfully.
4. Save the current control effort and error signal for future purpose.

Steps 1 and 3 of the DC motor speed control were already completed in the two previous sections. In this section, we will focus on the digital controller design and implementation. A schematic diagram to illustrate *closing the control loop with the unity feedback* for the DC motor is given in Fig. 4.

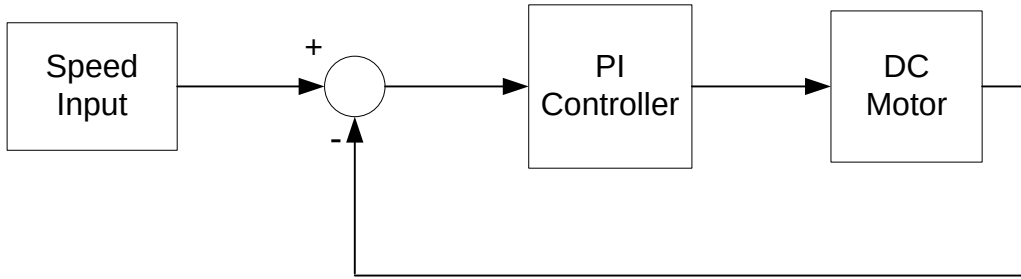


Fig. 4. Closed-loop control of a DC motor

From (5), the transfer function of the DC motor system is

$$G(s) = \frac{W(s)}{V(s)} = \frac{K_t}{RJs + K_t K_b} \quad (6)$$

Using the parameters given in the DC motor datasheet, the plant transfer function is obtained as

$$G(s) = \frac{20.2}{9.92s + 2.57} \quad (7)$$

A proportional-integral (PI) controller can be designed using a root locus as (refer to Franklin and Powell)

$$D(s) = \frac{K_p s + K_I}{s} = \frac{1.5s + 5}{s}, \quad (8)$$

where $K_p = 1.5$ V-s/rad and $K_I = 5$ V.

The root locus of the closed-loop system is shown in Fig. 5. The damping ratio is 0.52 and system natural frequency is 3.19 rad/s. With sampling period of $h = 0.209$ s, the following discrete-time controller was obtained using the MATLAB “c2d” command based on the Tustin’s method.

$$\begin{aligned} u(k) &= u(k-1) + [K_p + 0.5K_I h]e(k) + [0.5K_I h - K_p]e(k-1) \\ &= u(k-1) + 2e(k) - e(k-1) \end{aligned} \quad (9)$$

As you can see, the continuous-time controller parameters $K_p = 1.5$ V-s/rad and $K_I = 5$ V were carefully chosen and the discrete-time controller parameters were rounded so that

no time-consuming floating-point arithmetic would be required in the control-effort calculation (9). On 16F877, it takes only 2 μs and 48 μs for a 16-bit integer addition and multiplication, respectively, whereas 32 μs and 147 μs for a floating-point addition and multiplication, respectively (*PIC MCU C Compiler Manual*, p. 226).

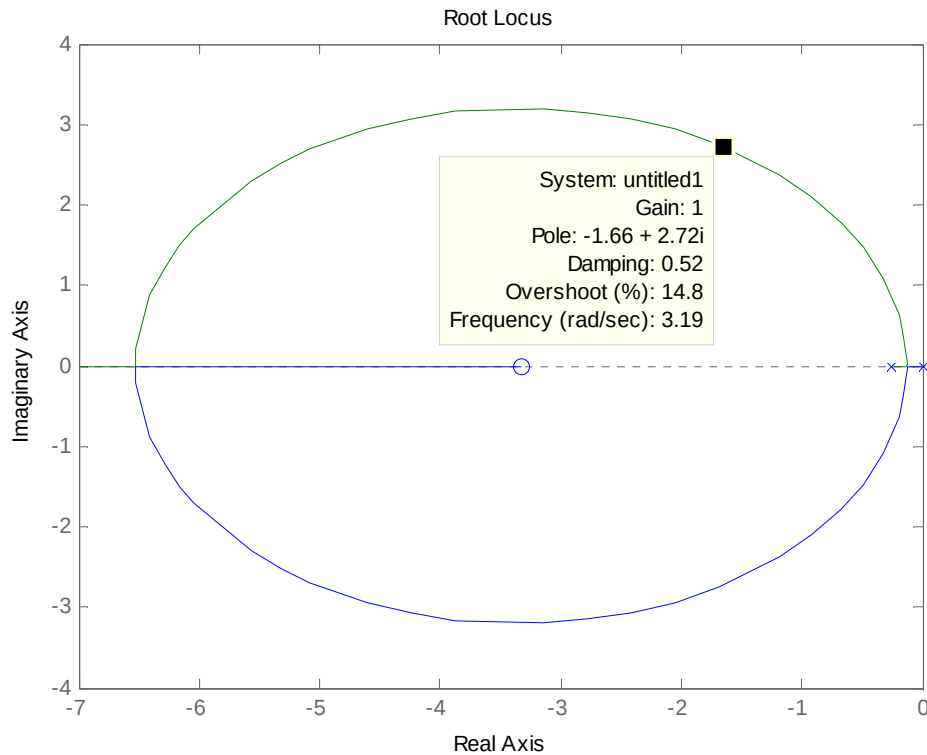


Fig. 5. Closed-loop system root locus

Assignment

Write this discrete-time control law (9) in C code.

5. Implementation of Closed Speed Control

Remove the potentiometer and complete the wiring connection as shown in Fig. 6. To set the ground reference voltage, connect the power ground and the signal ground at a single point. Double-check the wiring connection before you turn on the power.

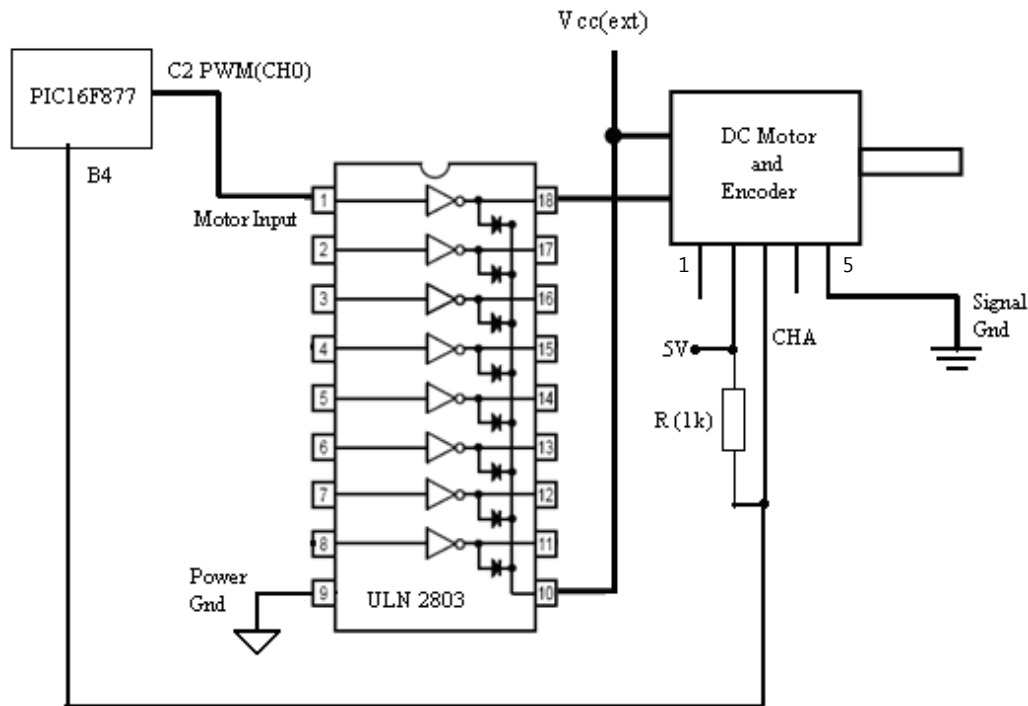


Fig. 6. Schematic diagram for DC motor closed-loop speed control

Assignment

Write and implement a C program to close the speed control loop of the DC motor system. Use the sample program given in Section 2 as a skeleton. Inside the interrupt-service routine (i.e., the C function, `Timer1_Interrupt()`) in this sample code, add your own codes in the following sequence:

1. Resolve the rotor speed with the encoder value.
2. Implement the discrete-time control law given in (9).
3. Generate PWM output.
4. Save data.

In addition, the user should be able to set the reference speed input. This information must be accepted in the `main()` function. Test your program and ensure the motor speed follows the reference speed input.

```
//-----LAB2_DATABASE-----//

#include <16F877.h>
#include <stdio.h>
#include <stdlib.h>
#include <delay.h>
#include <rs232.h>

// Maintain database of 12 users and their dates of birth.
// Each has a user id number (01-12) and asks for their DOB
// Uses fictitious dates of birth
// 5-character password should be implemented for system admin with user id "00"
// Admin can see DOBs and calculate average age of the group in years, months, and days
// Each month will have 30 days

void main()
{
    //int A [12][4];
    //A[0][0] = 5;
    //printf("%d", A[0][0]);

    int ok = 0;
    int day, month, year, m1, m2, d1, d2, y1, y2, y3, y4;
    int id1, id2, id;
    int pi1 = 0, pi2 = 0, pi3 = 0, pi4 = 0, pi5 = 0;
    int pw1 = 1, pw2 = 2, pw3 = 3, pw4 = 4, pw5 = 5;
    int u1, u2, u;
    int answer;
    int DOB[13][4];
    int whileprog = 1, whileid = 1, whileadmin = 1;
    int adminchoice = 0;
    int i;
    long avgmonths = 0;
    long avgdays = 0;
    long avgyears = 0;

    // prompt user input, id number
    // if 01-12, ask for year first (input year), then month
    // (input month), then day (input day)
    // Store into year, month, and day arrays
    // if 00, ask for password. If incorrect, report error and start over
    // if not 00-12, report error and ask for input again
    //

    while (1){
        while (whileid == 1){//id number while loop
            printf("Please enter your user id number\n\n");
            id1 = getch() - 48;
            printf("%d", id1);
            id2 = getch() - 48;
            printf("%d\n\n", id2);
            id = 10 * id1 + id2;
            if (id >= 1 && id <= 12){
                whileid = 0;
            }
            else{
                if (id == 0){
                    whileid = 0;
                }
            }
        }
    }
}
```

```

        else{
            printf("You have entered an incorrect id.  
Please try again.\n\r");
            delay_ms(1000);
        }
    }
} //id number while loop
if (id == 00){
    printf("Hello Administrator! Please enter your  
5-digit password.\n\r");
    pi1 = getc() - 48;
    printf("%d", pi1);
    pi2 = getc() - 48;
    printf("%d", pi2);
    pi3 = getc() - 48;
    printf("%d", pi3);
    pi4 = getc() - 48;
    printf("%d", pi4);
    pi5 = getc() - 48;
    printf("%d\n\r", pi5);
}
if (id >= 1 && id <= 12){
    printf("Thank you. Please enter your birth date in MMDDYY format\n\r");
    printf("Example 122091 = December 20th, 1991\n\r");
    m1 = getc() - 48;
    printf("%d", m1);
    m2 = getc() - 48;
    printf("%d", m2);
    d1 = getc() - 48;
    printf("%d", d1);
    d2 = getc() - 48;
    printf("%d", d2);
    //y1 = getc()-48;
    //printf("%d", y1);
    //y2 = getc()-48;
    //printf("%d", y2);
    y3 = getc() - 48;
    printf("%d", y3);
    y4 = getc() - 48;
    printf("%d\n\r", y4);
    month = 10 * m1 + m2;
    day = 10 * d1 + d2;
    year = 10 * y3 + y4;

    DOB[id][0] = id;
    DOB[id][1] = month;
    DOB[id][2] = day;
    DOB[id][3] = year;

    printf("Thank you. Next user please.\n\r");
}
while (whileadmin == 1){ //admin while loop
    if (pi1 == pw1&&pi2 == pw2&&pi3 == pw3&&pi4 ==  
pw4&&pi5 == pw5){

        printf("What would you like to see?\r\n");
        printf("1-all users DOBs\n\r 2-one user DOB\n\r  
3-Average user age\n\r");
        adminchoice = getc() - 48;

        if (adminchoice == 1){
            for (i = 1; i <= 12; i++){
                printf("User %d DOB is %d %d %d\n\r", i,  
DOB[i][1], DOB[i][2], DOB[i][3]);
            } //forloop
        } //choice1
    }
}

```

```

        if (adminchoice == 2){
            printf("Which user would you like to see?\n\r");
            u1 = getc() - 48;
            printf("%d", u1);
            u2 = getc() - 48;
            printf("%d\n\r", u2);
            u = 10 * u1 + u2;
            printf("User DOB is \n\r");
            printf("%d %d %d", month, day, year);
        }//choice2

        if (adminchoice == 3){
            for (i = 1; i <= 12; i++){
                avgmonths = avgmonths + (DOB[i][1]);
                avgdays = avgdays + (DOB[i][2]);
                avgyears = avgyears + (DOB[i][3]);
            }//forloop
            printf("The avg. year is %lu years, month is %lu and day
                is %lu \n\r", avgyears / 12, avgmonths / 12, avgdays / 12);
        }//choice3
        avgmonths = 0;
        avgdays = 0;
        avgyears = 0;

        printf("Would you like to see more? Y or N\n\r");
        answer = getc();
        if (answer == 89 || answer == 121){
            printf("OK\n\r");
        }
        if (answer == 78 || answer == 110){
            printf("Thank you for your service!\n\r\n\r");
            whileadmin = 0;
            pi1 = 0, pi2 = 0, pi3 = 0, pi4 = 0, pi5 = 0;
        }

        }//if password is correct
    }else{
        whileadmin = 0;
        pi1 = 0, pi2 = 0, pi3 = 0, pi4 = 0, pi5 = 0;
    }//else
} //admin while loop
whileid = 1; whileadmin = 1;
} //program while loop
} //main

//-----LAB3_ADC-----

#include <16F877.h>
#define ADC = 10
#define delay(clock = 1000000)
#define rs232(baud = 9600, xmit = PIN_C6, rcv = PIN_C7)

main(){
    long adc_value;
    float volt;
    setup_adc_ports(RA0_ANALOG);
    setup_adc(ADC_CLOCK_INTERNAL);

    while (1){
        set_adc_channel(0);
        delay_us(10);
        adc_value = read_adc();
        volt = adc_value * 5 / 1023.0;
    } //END WHILE
}

```



```
//END MAIN
```

```
//-----LAB4_ASS-----
```

```
#include <16F877.h>
#use delay(clock = 1000000)
#use rs232(baud = 9600, xmit = PIN_C6, rcv = PIN_C7)
short flag = 1;
```

```
#priority EXT, TIMER1, RDA
```

```
#INT_TIMER1
```

```
Timer1_Interrupt()
{
    if (flag){
        output_high(PIN_B2);
        printf("Timer Interrupt !\n\r");
    }
    else
        output_low(PIN_B2);
    flag = ~flag;
}
```

```
#INT_EXT
```

```
External_Interrupt()
{
    printf("External Interrupt !\n\r");
    output_high(PIN_B2);
    delay_ms(500);
    output_low(PIN_B2);
}
```

```
#INT_RDA
```

```
RS232C_Receive_Interrupt()
{
    char a;
    printf("RS232c Receive Interrupt !\n\r");
    a = getc();
    printf("Received data (ASCII) = %u\n\r", a);
    output_high(PIN_B2);
    delay_ms(500);
    output_low(PIN_B2);
}
```

```
main()
```

```
{
    ext_int_edge(H_to_L);
    enable_interrupts(INT_EXT);
    enable_interrupts(INT_RDA);
    setup_timer_1(T1_INTERNAL | T1_DIV_BY_8);
    // setup timer1
    // clock source = internal = clock/4 = 2.5MHz
    // overflow at every 1 / 2.5 MHz * 2^16 = 0.0262 s
    // prescaler = 8
    // interrupt every 0.0262 * 8 = 0.209 s
    enable_interrupts(INT_TIMER1);
    enable_interrupts(GLOBAL);

    while (1) {
        printf("...\n\r");
        delay_ms(100);
    }
}
```

```
//-----LAB4_EXTINT-----
```

```
#include <16F877.h>
```

```
#use delay(clock = 1000000)
#use rs232(baud = 9600, xmit = PIN_C6, rcv = PIN_C7)

#INT_EXT
External_Interrupt()
{
    printf("External Interrupt !\n\r");
    output_high(PIN_B2);
    delay_ms(500);
    output_low(PIN_B2);
}

main()
{
    ext_int_edge(L_to_H);
    enable_interrupts(INT_EXT);
    enable_interrupts(GLOBAL);
    while (1)
        printf("...\n\r");
}

//-----LAB4RECINT-----
#include <16F877.h>
#use delay(clock = 1000000)
#use rs232(baud = 9600, xmit = PIN_C6, rcv = PIN_C7)

#INT_RDA
RS232C_Receive_Interrupt()
{
    char a;
    printf("RS232c Receive Interrupt !\n\r");
    a = getc();
    printf("Received data (ASCII) = %u\n\r", a);
    output_high(PIN_B2);
    delay_ms(500);
    output_low(PIN_B2);
}

main()
{
    enable_interrupts(INT_RDA);
    enable_interrupts(GLOBAL);
    while (1)
        printf("...\n\r");
}

//-----LAB4TIMER-----
#include <16F877.h>
#use delay(clock = 1000000)
#use rs232(baud = 9600, xmit = PIN_C6, rcv = PIN_C7)
short flag = 1;

#INT_TIMER1
Timer1_Interrupt()
{
    if (flag)
        output_high(PIN_B2);
    else
        output_low(PIN_B2);
    flag = ~flag;
}

main()
{
    setup_timer_1(T1_INTERNAL | T1_DIV_BY_8);
    // setup timer1
    // clock source = internal = clock/4 = 2.5MHz
    // overflow at every 1 / 2.5 MHz * 2^16 = 0.0262 s
    // prescaler = 8
```

```

    // interrupt every 0.0262 * 8 = 0.209 s
    enable_interrupts(INT_TIMER1);
    enable_interrupts(GLOBAL);

    while (1)
        printf("...\n\r");
}

//-----LAB6-----
#include <16F877.h>
#define delay(clock = 1000000)
#define rs232(baud = 9600, xmit = PIN_C6, rcv = PIN_C7)
main()
{
    set_tris_b(0xFF);
    int count = 0;
    while (1) {

        delay_ms(5)
        if (input(PIN_B4)){
            if (input(PIN_B5)){
                // printf("A \n\r");
                // output_high(PIN_A0);
                count = count + 1;
                printf("counter value is %d", count);
            }

        }
        //else
        // output_low(PIN_A0);
        delay_ms(5)
        if (input(PIN_B5)) {
            if (input(PIN_B4)) {
                // printf("B \n\r");
                // output_high(PIN_A1);
                count = count - 1;
                printf("counter value is %d", count);
            }

        }
        //else
        //output_low(PIN_A1);

    } // close initial while
} // close main

//-----LAB8ASS1-----
#include <16F877.h>
#define device ADC = 10
#define delay(clock = 1000000)
#define rs232(baud = 9600, xmit = PIN_C6, rcv = PIN_C7)

main(){
    long adc_value;
    float volt;
    setup_adc_ports(RA0_ANALOG);
    setup_adc(ADC_CLOCK_INTERNAL);

    while (1){
        set_adc_channel(0);
        delay_us(10);
        adc_value = read_adc();
        volt = adc_value * 5 / 1023.0;
        delay_500ms;
        printf("%d "volts" / n / r);
    }
}

```

```

    }//END WHILE

}

} //END MAIN

//-----LAB9-----

#include <16F877.h>
#define ADC = 10
#define delay(clock = 1000000)
#define rs232(baud = 9600, xmit = PIN_C6, rcv = PIN_C7)
main()
{
    char selection;
    long value;

    printf("\r\nFrequency:\r\n");
    printf(" 1) 9.75 khz\r\n");
    printf(" 2) 2.45 khz\r\n");
    printf(" 3) 0.6 khz\r\n");
    do {
        selection = getc();
    } while ((selection<'1') || (selection>'3'));

    setup_ccp1(CCP_PWM);
    switch (selection) {
        case '1': setup_timer_2(T2_DIV_BY_1, 255, 1);
            break;
        case '2': setup_timer_2(T2_DIV_BY_4, 255, 1);
            break;
        case '3': setup_timer_2(T2_DIV_BY_16, 255, 1);
            break;
    }

    setup_port_a(ALL_ANALOG);
    setup_adc(adc_clock_internal);
    set_adc_channel(0);
    printf("%c\r\n", selection);

    while (1) {
        value = read_adc();
        printf("PWM Command = %lu\r\n", value);
        set_pwm1_duty(value);
    }
}

//-----LAB10-----

#include <16F877.h>
#define ADC = 10
#define delay(clock = 1000000)
#define rs232(baud = 9600, xmit = PIN_C6, rcv = PIN_C7)

long cnt = 0;
long speed = 0;
long value;

#define INT_TIMER1
Timer1_Interruption()
{
    printf("PWM Command = %lu\r\n", value);

    speed = cnt*0.00552;
    printf("Speed of DC motor is %ld rps. \r\n", speed);
}

```

```

    cnt = 0;
}
main()
{
    int b1, b2;
    /* char selection;
    printf("\r\nFrequency:\r\n");
    printf(" 1) 9.75 khz\r\n");
    printf(" 2) 2.45 khz\r\n");
    printf(" 3) 0.6 khz\r\n");
    do {
        selection=getc();
    } while ((selection<'1') || (selection>'3'));

    setup_ccp1(CCP_PWM);
    switch (selection) {
        case '1' : setup_timer_2(T2_DIV_BY_1, 255, 1);
        break;
        case '2' : setup_timer_2(T2_DIV_BY_4, 255, 1);
        break;
        case '3' : setup_timer_2(T2_DIV_BY_16, 255, 1);
        break;
    }

    setup_port_a(ALL_ANALOG);
    setup_adc(adc_clock_internal);
    set_adc_channel(0);
    printf("%c\r\n",selection);
    */

    setup_timer_1(T1_INTERNAL | T1_DIV_BY_8);
    enable_interrupts(INT_TIMER1);
    enable_interrupts(GLOBAL);

    while (1) {
        value = read_adc();
        set_pwm1_duty(value);

        if (input(PIN_B4))
            b1 = 1;
        else
            b1 = 0;
        if (b1 != b2)
            cnt++;
        b2 = b1;
    }
}

//-----MEEN667HW10-----

unsigned char GetByte(void)
{
    // reads a 8bit serial temperature signal from PIN_B0
    and returns 8bit char
    int i; int bits;
    int number = 0;
    char c;

    StartConvertT()           // begin temp conversion
        StopConvertT()
        ReadTemperature();     // start reading register

    for i = 0:8 {
        output_high(CLK)

```

```
        output_low(CLK)          // toggle clock to falling edge

        bits[i + 1] = input(PIN_B0);    // reads 1bit of output
        number = number + bits[i + 1] * 2 ^ i; // convert binary to decimal
    }

    c = (char)number;                //store temp as char
    return c
}

void PutByte(unsigned char data)
{
    // receives a 8-bit char, writes a 8-bit serial signal
    // through PIN_B0, and returns nothing

    num = (int)data

    // convert to binary
    bin[1..9] = 0
    for digit = 1:9
        bin[digit] = num % 2
        num = num / 2
    end

    WriteConfig()
    // send 8 bites cereal
    for i = 0:8 {
        output_low(CLK)
        output_high(CLK)          // toggle clock to rising edge

        bin[i + 1] = output(PIN_B0);    // sends 1 bit
    }
}
```